

VERIDION

Enterprise Agentic AI Platform for Vendor & Procurement Management

Complete End-to-End Workflow & Architecture — PostgreSQL Edition

Includes the Context Gate confidence-weighted decision engine, full database schema, real-time vendor risk scoring, legitimate real-world variance handling, the corrected Vendor / Company Profile / Platform Admin interface structure, automated vendor voice notifications, and the 6-month build plan.

Table of Contents

- Part 1 — System Architecture
- Part 2 — PostgreSQL Database Schema
 - 2.1 Identity & Access
 - 2.2 Procurement Lifecycle Core
 - 2.3 AI & Context Gate Layer
 - 2.4 Communication & Payments
 - 2.5 File Storage — Kept Inside PostgreSQL
 - 2.6 Neo4j — runs alongside PostgreSQL
- Part 3 — The Context Gate, In Detail
 - 3.1 What each agent must output
 - 3.2 The weighting formula
 - 3.3 Where else the gate applies
 - 3.3.1 Worked example — a new vendor undercutting an established one
 - 3.4 The feedback loop
 - 3.5 Why a formula, not a trained model
 - 3.6 Displaying confidence to the user, and who actually decides
- Part 4 — Complete End-to-End Workflow
 - Phase A — Onboarding
 - Phase B — Requirement to Purchase Order
 - Phase C — Delivery & Invoicing
 - Phase D — AI Verification Pipeline
 - Phase E — Dispute Resolution
 - Phase F — Payment
 - Phase G — Continuous Learning & Reporting
 - Phase H — Project Completion Steps (Months 5-6)
- Part 5 — Vendor Risk Scoring Engine (Agent 5): Real-Time Update Logic
 - 5.1 When the score updates — the three triggers
 - 5.2 The update formula
 - 5.3 Worked example — a brand-new vendor's first delivery
 - 5.4 Worked example — an established vendor misses a delivery
 - 5.5 Agent 5 — pseudocode
 - 5.6 How this connects to the Context Gate
 - 5.7 Where vendor history comes from — no prefilling, by design
 - 5.7.1 How a Company Admin actually does this
 - 5.7.2 Worked example — real activity after a legacy import
 - 5.8 A platform-wide view — read-only, verified-only, for humans not the gate
- Part 6 — Agent Inventory: What We're Actually Building
 - 6.1 The original 9 agents (from the initial proposal)
 - 6.2 The 8 agents actually being built, mapped to the original 9

- 6.3 Where the Context Gate runs — suppliers vs. consumers
 - 6.4 Agent 5 vs Agent 8 vs Agent 6 — the exact relationship
 - 6.5 Why the 9 agents are still needed even though the PO is system-generated
- Part 7 — Handling Legitimate Real-World Variance (Without Weakening Fraud Detection)
 - 7.1 Partial delivery — match against the GRN, not the PO
 - 7.1.1 Optional: capturing an expected date for the remaining quantity
 - 7.2 Tax recalculated at invoice time — validate against current rates
 - 7.3 Multiple partial invoices — cumulative tracking on the PO
 - 7.4 Why fraud detection stays just as strict
 - 7.5 Updated schema changes required
- Part 8 — System Interfaces: Roles, Purpose & Capabilities
 - 8.1 Vendor Interface
 - 8.2 Company Profile Interface
 - 8.2.1 Company Admin
 - 8.2.2 Procurement role
 - 8.2.3 Finance role
 - 8.2.4 Warehouse role — detailed
 - 8.3 Platform Admin Interface
 - 8.4 Registration & Login — exactly what's needed, no more
 - 8.5 Required table changes for this restructuring
- Part 9 — Automated Vendor Notification via Voice Call
 - 9.1 Trigger point
 - 9.2 Provider choice — OmniDimension
 - 9.3 Required configuration (API keys)
 - 9.4 Message content
 - 9.5 Non-blocking design rule
 - 9.6 Schema additions
- Summary

Part 1 — System Architecture

Veridion is built as four layers: a shared React client, a Node.js/Express API layer, a PostgreSQL database for all structured and document data, and an AI service layer of eight specialised agents coordinated by RabbitMQ and grounded by a Neo4j graph for vendor relationship history.



RabbitMQ's role: once OCR finishes reading an invoice, four agents — Matching, Compliance, Fraud Detection, and Vendor Risk — all need to run, and none of them depend on each other. RabbitMQ broadcasts the OCR result to all four simultaneously instead of running them one after another, which is the platform's clearest demonstration of real distributed-systems design.

Part 2 — PostgreSQL Database Schema

Nineteen tables, organised into five functional groups. Flexible AI outputs (structured OCR data, compliance details, the Context Gate's agent inputs and weights) are stored in JSONB columns, giving document-store flexibility inside a fully relational, foreign-key-enforced schema.

2.1 Identity & Access

Table	Key columns
users	id, name, email, password_hash, role (vendor/procurement/finance/admin), company_id (FK, nullable), created_at
companies	id, name, gstin, address, industry_type
vendors	id, user_id (FK), company_name, gstin, pan, business_reg_proof_path, bank_account_number, bank_ifsc, verification_status, trust_score, phone_number, preferred_notification_channel ('voice_call'/'sms'/'whatsapp'/'app_only', default

	'sms'), created_at
roles_permissions	role, permission_key

2.2 Procurement Lifecycle Core

Table	Key columns
requirements	id, company_id (FK), title, description, category, quantity, unit, deadline, status, created_by (FK), created_at
quotations	id, requirement_id (FK), vendor_id (FK), price, delivery_days, notes, ai_rank_score, ai_rank_reasoning, status, submitted_at
purchase_orders	id, requirement_id (FK), quotation_id (FK), vendor_id (FK), company_id (FK), agreed_price, agreed_quantity, agreed_delivery_date, status, po_document_path, created_at
goods_receipt_notes	id, po_id (FK), received_quantity, received_date, warehouse_notes, recorded_by (FK), discrepancy_flag
invoices	id, po_id (FK), grn_id (FK), vendor_id (FK), invoice_number, invoice_amount, gst_amount, gstin_on_invoice, invoice_file_path, status, due_date, submitted_at

2.3 AI & Context Gate Layer

Table	Key columns
document_extractions	id, invoice_id (FK), raw_ocr_text, structured_data JSONB, confidence_score, bounding_boxes JSONB
compliance_checks	id, invoice_id (FK), gst_valid, gst_details JSONB, issues_found text[], confidence_score, data_volume
matching_results	id, invoice_id (FK), po_match_score, grn_match_score, mismatch_fields JSONB, overall_match, confidence_score, data_volume
fraud_flags	id, vendor_id (FK), invoice_id (FK, nullable), flag_type, severity, evidence JSONB, confidence_score, resolved
vendor_risk_scores	id, company_id (FK), vendor_id (FK), score, on_time_delivery_pct, dispute_rate, invoice_accuracy_pct, data_volume, data_source ('platform_verified'/'self_reported', default 'platform_verified'), platform_verified_event_count, last_updated — UNIQUE (company_id, vendor_id), see Part 5.7.1
decisions	id, invoice_id (FK), agent_inputs JSONB, gate_weights JSONB, final_score, final_decision, reasoning_text, decided_at
agent_overrides	id, decision_id (FK), field_name, original_value, corrected_value, overridden_by (FK), overridden_at

The decisions table is where the Context Gate's output lives — agent_inputs stores each agent's verdict, confidence, and data volume; gate_weights stores the normalised weight the gate assigned to each agent for that specific decision; reasoning_text is the plain-English explanation rendered on the Invoice Review page.

2.4 Communication & Payments

Table	Key columns
vendor_communications	id, invoice_id (FK), draft_text, status (pending_send/sent), mismatch_type, sent_by (FK), sent_at, resolved, resolution_time_hours
notifications	id, user_id (FK), message, type, read, created_at
payments	id, invoice_id (FK), amount, status, approved_by (FK), paid_at, payment_reference
payment_reminders	id, invoice_id (FK), reminder_sent_at, reminder_type
outbound_notifications	id, vendor_id (FK), reference_type ('quotation_accepted'), reference_id (FK to purchase_orders), channel ('voice_call'/'sms'/'whatsapp'), status ('queued'/'sent'/'failed'), provider_message_id, sent_at — see Part 9

2.5 File Storage — Kept Inside PostgreSQL

Rather than GridFS or a separate object store (S3/MinIO), files are handled the simplest way that still keeps everything in one system:

- Uploaded files (invoices, POs, business registration proofs) are saved to a Docker-mounted disk volume.
- Each table that references a file stores only the file's path as a text column (business_reg_proof_path, po_document_path, invoice_file_path) — shown above.
- For small supporting files (thumbnails, generated PDFs under a few MB) a bytea column in a dedicated documents table is a fine alternative if the team prefers everything queryable from SQL directly.
- Either approach avoids standing up a second storage system, while keeping full relational integrity between a file and the record it belongs to.

2.6 Neo4j — runs alongside PostgreSQL

Graph queries do not belong in a relational database. Neo4j remains a separate store purely for vendor relationship modelling and fraud pattern detection.

- Nodes: Vendor, Invoice, PurchaseOrder, Company, BankAccount, Address
- Relationships: SUPPLIED_TO, PAID_BY, SHARES_BANK_ACCOUNT_WITH, SHARES_ADDRESS_WITH, DISPUTED_WITH
- Powers: shell-company detection (shared bank account/address across “different” vendors) and split-billing detection (multiple invoices against one PO, timed close together, each just under the auto-approval threshold)

Part 3 — The Context Gate, In Detail

This is the mechanism that turns the Decision Engine from a fixed-rule aggregator into Veridion's actual novel contribution.

3.1 What each agent must output

Every agent feeding the Decision Engine returns three things, not one:

- verdict — the actual result (match/no-match, valid/invalid, flagged/clear, a risk score)
- confidence — how sure the agent is (0–1), based on its own signal quality
- data_volume — how much history informed the verdict (e.g. vendor's past transaction count, OCR scan clarity, number of comparable line items matched)

3.2 The weighting formula

```
raw_weight_i = confidence_i x log(1 + data_volume_i)
gate_weight_i = raw_weight_i / SUM(raw_weight_all_agents) // normalised to 1.0
final_score = SUM(gate_weight_i x agent_verdict_score_i)
```

Worked example — a blurry-scan invoice from a new vendor:

- Matching agent: “mismatch found”, confidence 0.35 (OCR was unreliable), data_volume = 3 line items
- Fraud agent: “no shell company match”, confidence 0.95 (Neo4j exact-match query), data_volume treated as 1
- Compliance agent: “GST valid”, confidence 0.9

- Risk agent: “elevated risk”, confidence 0.4 (only 1 past transaction, data_volume = 1)

The gate down-weights the shaky Matching signal and the thin Risk signal, letting the high-confidence Fraud and Compliance checks dominate — so a clean transaction with a poor-quality scan isn't over-flagged. That reasoning is written directly into decisions.reasoning_text and shown on the Invoice Review page.

3.3 Where else the gate applies

The same formula is reused for Quotation Ranking (Agent 6) — weighing Price, Delivery Fit, and Past Performance by how much history a vendor has, so a brand-new vendor's thin performance record doesn't get averaged in as if it were reliable data. Everywhere else in the pipeline, agents remain independent; the gate is used only at the two points where multiple signals must be fused into a single human-facing decision.

3.3.1 Worked example — a new vendor undercutting an established one

Two vendors quote on the same requirement: Vendor A (established, 50 past transactions) at ₹520/unit with a 96% on-time record; Vendor B (brand new, 0 past transactions) at ₹460/unit, with no history at all. Price is always known with full certainty the moment a quote is submitted, so it keeps a meaningful baseline weight for both vendors regardless of history. Past Performance is the factor that scales hard with data_volume — for Vendor B it collapses toward zero, and that freed-up weight flows onto Price and Delivery instead, the two things that are fully known right now.

	Vendor A (established)	Vendor B (new)
Price weight	~14%	~67%
Delivery weight	~13%	~33%
Past Performance weight	~73%	0%

Case 1 — the price gap is real (₹460 vs ₹520):

Vendor B's final score comes out higher. Since Price carries two-thirds of B's total weight and B is genuinely cheaper, that advantage comes through strongly enough to beat A's 73%-weighted track record. The new vendor wins the ranking, fairly — on the strength of something fully verifiable today, not despite having no history.

Case 2 — the price gap is thin (₹510 vs ₹520):

B's small price edge barely moves their score, because their weighting is concentrated on Price and Delivery but neither is decisive at this gap. A's dominant weight on a genuinely strong 96% record stays untouched and wins. A new vendor does not win by default just for being cheap — only when the price advantage is large enough to outweigh what is actually known.

The gate is not asking “who has more evidence, therefore who wins” — it is asking “which of this vendor's known strengths should be trusted, and how much.” This is also not a permanent disadvantage for new vendors: the moment Vendor B completes their first delivery, data_volume becomes 1 and Past Performance stops being zero-weighted, growing from there exactly as described in Part 5.

The same zero-weighting applies from the other direction, too: a brand-new company evaluating any vendor — even one with a long history elsewhere on the platform — starts from data_volume = 0 in its own private vendor_risk_scores row, and the gate weighs that company's ranking accordingly. It does not borrow confidence from other companies' experience with that vendor; see Part 5.8 for exactly what is and is not shared across companies.

3.4 The feedback loop

- Every Finance Officer override of a decision (flagged→approved or vice versa) is logged in agent_overrides.

- Monthly, an aggregation query reviews which agent's weight was highest in overturned decisions.
- If one agent is consistently over-weighted relative to how often it was actually right, its confidence-scaling constants are manually adjusted by the admin — a transparent, explainable tuning process, not an opaque auto-tuning model, which is the right level of ambition for a 6-month build.

3.5 Why a formula, not a trained model

Gating mechanisms of this kind are not unique to Veridion — a related project (EmoSync, a multimodal emotion-recognition system) uses a similarly named “Context Gate” to decide how much to trust each input modality (vision, audio, text) when predicting an emotion. Both gates share the same underlying idea: weigh multiple sources of information by how reliable each one is for this specific input, rather than trusting all of them equally. The two implementations differ deliberately, and the difference is worth stating explicitly.

	EmoSync's Context Gate	Veridion's Context Gate
What it gates	Input modalities (vision, audio, text)	AI agent opinions (Matching, Compliance, Fraud, Vendor Risk / Price, Delivery, Past Performance)
How weights are computed	A trained 3-layer MLP, learned end-to-end from labelled data with modality dropout	A fixed formula: $\text{confidence} \times \log(1 + \text{data_volume})$, normalised — no training involved
Requires training data?	Yes — CMU-MOSI/MOSEI/IEMOCAP provide thousands of labelled examples	No — a new platform has zero historical transactions on day one; there is nothing to train a gate on yet
Explainability	Learned weights are a black box; a separate attribution method (e.g. SHAP) would be needed to explain a routing decision	The formula itself is the explanation — confidence and data_volume are read directly into decisions.reasoning_text
Consequence of a wrong weight	A research metric (accuracy on a benchmark) moves	Real money moves, or a vendor's trust is misjudged — a materially higher-stakes error

This is a deliberate design choice, not a simplification made for lack of time. A trained gate needs a labelled dataset of “correct” weightings to learn from, which does not exist for a brand-new platform; a transparent formula can be reasoned about, audited, and explained to a non-technical Finance reviewer without any additional tooling; and the cost of a misweighted decision is financial rather than a benchmark score, which favours a mechanism that is predictable and inspectable over one that is merely accurate.

The two designs converge naturally over time rather than being permanently different approaches: once agent_overrides (Section 3.4) has accumulated months of real examples of correct and incorrect gate weightings, that is exactly the labelled dataset a trained gate would need. At that point, replacing the fixed formula with a learned model — architecturally closer to EmoSync's approach — becomes a realistic next step, rather than something attempted before there is any data to learn from.

3.6 Displaying confidence to the user, and who actually decides

Two things follow directly from how the gate works, and both are worth stating explicitly rather than leaving implicit in the schema.

The system's confidence is shown, not just its verdict.

decisions.reasoning_text and gate_weights (Part 2.3) are not just stored for audit purposes — they render as a visible confidence indicator directly beside the approve/flag decision on the Invoice Review page, and beside each vendor's rank on the Quotation Comparison page. A short badge (e.g. “System confidence: High / Medium / Low”, derived from the gate's aggregate confidence across contributing agents) sits next to the decision itself, with the full plain-English reasoning available on expansion — so a Finance Officer or Procurement user sees at a glance how sure the system is, not only what it concluded.

The AI recommends; the company always decides.

Vendor selection is never made by the system. Agent 6 ranks quotations and explains its reasoning, but Procurement can accept the top-ranked vendor or select a different one entirely (Part 8.2.2) — this does not change based on the gate's confidence or the price gap involved. The same principle holds for invoice decisions: even a high-confidence auto-approval remains visible and reversible by Finance, and every flagged case explicitly routes to a human by design (Part 4, Phase D). Confidence changes how strongly the system recommends; it never removes the human's final choice.

Part 4 — Complete End-to-End Workflow

From first login to a completed, paid transaction.

Phase A — Onboarding

Step 1. Vendor registration

Vendor fills out the registration form, uploads GST/PAN/business registration proof to disk storage, and a vendors row is created with `verification_status = 'pending'`.

Step 2. Automated verification checks

Backend runs format validation (regex/checksum on GSTIN/PAN), a sandbox/mock GSTN verification call, and an AI-assisted document authenticity check comparing the uploaded proof against the entered GSTIN/PAN. Results are attached to the vendor record.

Step 3. Admin approval

Admin reviews the verification queue page, sees the automated results, and approves or rejects. On approval, `verification_status = 'verified'` and the vendor can act on the platform — deliberately kept human-gated.

Phase B — Requirement to Purchase Order

Step 4. Requirement posted

Step 5. Quotations submitted

Step 6. AI ranking — Context Gate applied here

Step 7. Vendor selected

Phase C — Delivery & Invoicing

Step 8. Goods received

Step 9. Invoice submitted

Phase D — AI Verification Pipeline (the core engine)

Step 10. OCR & Structuring

Step 11. Parallel agent fan-out

Step 12. Context Gate — Decision Engine

Step 13a. Clean path

Step 13b. Flagged path

Step 13c. Suspicious path

Step 14. Invoice Review page

Phase E — Dispute Resolution (if applicable)

Step 15. Review and send

Step 16. Vendor responds

Step 17. Resolution recorded

Phase F — Payment

Step 18. Payment approved

Step 19. Reminders scheduled

Step 20. Vendor tracks status

Phase G — Continuous Learning & Reporting

Step 21. Overrides logged

Step 22. Monthly tuning review

Step 23. Dashboards

Phase H — Project Completion Steps (Months 5–6)

- Seed a synthetic dataset (clean transactions, deliberately mismatched ones, 2–3 planted fraud patterns) into PostgreSQL.
- Run the full Step 1–23 flow against synthetic data for every path — auto-approve, flagged/disputed, fraud-suspicious.
- Compute evaluation numbers: % correctly auto-approved, % correctly flagged, and the gate's decision accuracy versus a fixed-equal-weight baseline (the ablation study).
- Write up the architecture, the Context Gate mechanism, and the evaluation results for the final report.
- Rehearse a live demo walking through one clean example and one flagged example end to end — mirroring the original BrightBuild / SteelCorp example.

Part 5 — Vendor Risk Scoring Engine (Agent 5): Real-Time Update Logic

The `vendor_risk_scores` table and Agent 5 were already defined in Part 2 and Part 4, but the exact logic for how and when a vendor's score actually updates was not — this section closes that gap so the agent can be implemented directly from this document.

5.1 When the score updates — the three triggers

The score is not recalculated on a schedule (e.g. nightly). It is event-triggered — it recalculates the moment one of three real events happens, and the new number is available immediately for the very next decision the system makes.

Trigger event	Metric affected	Fires when
GRN confirmed	on_time_delivery_pct	Warehouse confirms goods received; received_date is compared against the PO's agreed_delivery_date
Dispute resolved	dispute_rate	A vendor_communications row is marked resolved
Invoice decision finalised	invoice_accuracy_pct	The Context Gate (Agent 8) writes a decisions row for that vendor's invoice

5.2 The update formula

Each metric is a running average, updated incrementally rather than recomputed from full history every time (cheaper, and the same result mathematically):

```
new_value      = (old_value * old_data_volume + this_event_value) / (old_data_volume + 1)
new_data_volume = old_data_volume + 1

-- this_event_value is 100 if the event was "good" (on-time, accurate, no
-- dispute) and 0 if it was "bad" (late, inaccurate, disputed)
```

data_volume increases by exactly 1 per triggering event, regardless of whether the outcome was good or bad — it tracks how much evidence exists, not how good the vendor is. That distinction matters: it is what the Context Gate reads separately from the score itself.

5.3 Worked example — a brand-new vendor's first delivery

Before this order: data_volume = 0, on_time_delivery_pct = null (no fabricated default — the system does not pretend to know).

The vendor delivers on time. The moment the GRN is confirmed:

```
new_on_time_pct = (0 x 0 + 100) / (0 + 1) = 100%
new_data_volume = 1
```

The stored score is now 100%, calculated immediately. But data_volume = 1 means the Context Gate's confidence-scaling term, $\log(1 + \text{data_volume})$, is still very small — so this score is barely trusted yet in any decision that uses it, even though the number itself looks perfect. One good delivery updates the score but does not yet earn full trust.

5.4 Worked example — an established vendor misses a delivery

Before this order: data_volume = 50, on_time_delivery_pct = 96% (48 on-time out of 50). This vendor misses the delivery window on the new order. The moment the GRN is confirmed late:

```
new_on_time_pct = (96 x 50 + 0) / 51 = 4800 / 51 ~= 94.1%
new_data_volume = 51
```

This also updates immediately — the same trigger, the same formula. Because 50 prior transactions back this score, one missed delivery only nudges it from 96% to 94.1%, not down to some cliff-edge “unreliable” flag. A vendor with a long track record is judged on the balance of their history, not on their most recent event alone.

5.5 Agent 5 — pseudocode

```
function onGrnConfirmed(grn, po):
  onTime = grn.received_date <= po.agreed_delivery_date
  updateVendorMetric(po.vendor_id, "on_time_delivery_pct", onTime ? 100 : 0)

function onDisputeResolved(communication):
  wasDisputed = true // this event itself is the "bad" outcome being counted
  updateVendorMetric(communication.vendor_id, "dispute_rate", wasDisputed ? 0 : 100)

function onInvoiceDecisionFinalised(decision, invoice):
  wasAccurate = decision.final_decision != "flagged"
  updateVendorMetric(invoice.vendor_id, "invoice_accuracy_pct", wasAccurate ? 100 : 0)

function updateVendorMetric(vendorId, metricName, eventValue):
  row = fetch vendor_risk_scores where vendor_id = vendorId
  newValue = (row[metricName] * row.data_volume + eventValue) / (row.data_volume + 1)
  newVolume = row.data_volume + 1
  update vendor_risk_scores set metricName = newValue, data_volume = newVolume,
    last_updated = now() where vendor_id = vendorId
  -- overall vendor score can then be recomputed as a simple weighted
  -- average of the three metrics, using the same table
```

5.6 How this connects to the Context Gate

Two separate numbers now exist for every vendor metric, and the gate uses them differently:

- The score itself (on_time_delivery_pct, etc.) — always current, updated the instant a triggering event occurs.
- data_volume — how much evidence backs that score, used only by the Context Gate's confidence term (Part 3, Section 3.2) to decide how much this metric should influence a final decision or ranking.

This is the mechanism that makes the earlier answer literal rather than conceptual: a new vendor's score is real, calculated, and live from their very first transaction — it simply is not yet trusted heavily, and that trust grows automatically, transaction by transaction, as data_volume increases. No separate “trust-building” logic is needed — it falls directly out of combining this update formula with the existing gate weighting formula.

5.7 Where vendor history comes from — no prefilling, by design

No agent is manually taught a vendor's history before deployment, and no vendor is ever seeded with fabricated data. Every vendor starts at data_volume = 0 for real — this is the same cold-start principle already established in Sections 5.3 and 5.6, stated here explicitly as a system-wide rule rather than something implied only by the worked examples. History accumulates automatically and exclusively from real events the platform itself witnesses: GRN confirmations, resolved disputes, and finalised invoice decisions (Section 5.1). Nothing about deploying the eight agents requires pre-loading vendor records — a newly deployed system and a newly registered vendor look identical, by construction.

The one legitimate exception: legacy import at onboarding.

A company migrating to Veridion often has genuine offline history with a vendor — years of paper-based transactions the platform never witnessed. Discarding that would be wasteful, but treating it as equivalent to platform-verified data would be dishonest. The resolution: Company Admin may optionally enter a starting data_volume and score for an existing vendor relationship, explicitly tagged data_source = 'self_reported' rather than 'platform_verified'. The entered transaction count is discounted once, at the moment of import — halved before being stored — rather than the Context Gate re-checking data_source on every future decision. That halved figure then becomes an ordinary starting point for the same running-average formula in Section 5.2: no special-casing is needed afterward, because the imported baseline naturally dilutes itself as real platform events accumulate on top of it, exactly like any older data point in a running average. data_source remains on the record permanently, but only as a visible audit label — a small badge shown wherever the score appears — not as something the gate recomputes differently each time.

Schema addition to vendor_risk_scores: + data_source ('platform_verified'/'self_reported', default 'platform_verified').

5.7.1 How a Company Admin actually does this

- The vendor must already exist on the platform — legacy import attaches to a real vendors row; it cannot create one. The vendor either self-registered already (Part 8.4), or the Company Admin invites them to register first.
- Company Admin opens the vendor's detail page from the Vendor Directory (already part of their oversight visibility, Part 8.2.1).
- An “Import legacy history” option appears only when the vendor currently has data_volume = 0 — no platform activity yet, in either direction. This is a one-time seeding action: the moment the vendor has any real platform history, the option disappears permanently, so a score can never be topped up later with a fresh unverifiable claim.
- The form: estimated number of past transactions, estimated on-time delivery %, dispute rate (optional), invoice accuracy % (optional), a short free-text justification, and a required confirmation checkbox — “I confirm this reflects our company's own records and has not been verified by Veridion.”
- On submit, the backend halves the entered transaction count, stores it as data_volume with data_source = 'self_reported', and the vendor_risk_scores row is created exactly as if it were the result of the update formula in Section 5.2 — every subsequent real event updates it the normal way.

Correction: this is not a Phase 3 problem, and not something to defer. Part 8.4 already lets any number of companies register on the same Veridion deployment, and nothing stops two companies from working with the same vendor. With no company_id on vendor_risk_scores, that is a single global row per vendor_id today — so Company A's experience with a vendor already silently affects what Company B sees about that same vendor, the moment a demo or deployment has two company accounts. The fix is cheap and worth making now rather than after real data has accumulated in a shared table.

company_id (FK) is added to vendor_risk_scores, with a UNIQUE (company_id, vendor_id) constraint — each company now builds its own private view of a vendor's reliability, based only on that company's own transactions with them. Agent 5's update functions (Section 5.5) resolve company_id from the triggering event (via po.company_id, or invoice → po → company_id) and scope every fetch and update to (company_id, vendor_id) instead of vendor_id alone. The legacy import flow in Section 5.7.1 fits this naturally, since a Company Admin's import was always meant to reflect only their own company's history — it should never have been able to influence any other company's view of that vendor. Agent 6 and Agent 8 read the score scoped to whichever company's decision is being made; this is a mechanical change to a lookup key, not a redesign of either agent.

This also makes Phase 3 of the original roadmap (cross-company vendor reputation) cleaner to build later: instead of retrofitting isolation onto data that has already been mixed together, Phase 3 becomes a deliberate, explicit feature — aggregating each company's separate, clean (company_id, vendor_id) records into a shared reputation view, with vendor consent, exactly as originally described. Consent-based sharing only makes sense if the underlying records were private to begin with.

5.7.2 Worked example — real activity after a legacy import

data_volume keeps changing after import, exactly the same way it would for a vendor with no legacy data at all — the formula does not know or care where a vendor's starting evidence came from, it simply treats it as prior evidence.

At import: the company reports 50 past transactions at 90% on-time. The system halves this before storing it:

```
data_volume = 25
on_time_delivery_pct = 90
data_source = 'self_reported'
```

The vendor then completes a real transaction on the platform — the GRN is confirmed, and this particular delivery is late. The same update formula from Section 5.2 fires, treating the imported row as just another starting point:

```
new_on_time_pct = (90 × 25 + 0) / (25 + 1) = 2250 / 26 ≈ 86.5%
new_data_volume = 25 + 1 = 26
```

data_volume moves from 25 to 26 and the score shifts from 90% to ~86.5%, using the identical mechanism applied to any vendor. data_source does not flip back to 'platform_verified' once real activity begins — it remains a permanent historical marker of how the vendor's record began, which stays honest about the fact that some of this vendor's evidence originated from an unverified company claim, even after real activity dilutes its influence over time.

Tracking how much evidence is real vs. imported.

Once real events start blending into an imported baseline, data_volume alone can no longer answer “of this vendor's total evidence, how much is a real platform transaction versus the imported estimate?” — a fair question for Finance or Platform Admin to want answered. A second counter, incremented only by real triggering events (GRN confirmed, dispute resolved, invoice decision finalised) and never at import time, closes this gap:

```
vendor_risk_scores
+ platform_verified_event_count (integer, default 0)
```

This lets the UI show a precise, honest breakdown — for example, “data_volume: 26 (25 legacy-imported, 1 platform-verified since joining)” — without changing the core formula, the Context Gate, or data_source's behaviour at all; it is purely an additional, transparent counter alongside the existing fields.

5.8 A platform-wide view — read-only, verified-only, for humans not the gate

Isolating each company's vendor_risk_scores row (Section 5.7.1's correction) is right for decision-making, but it creates a real gap on its own: a brand-new company with zero history has no way to judge an unfamiliar vendor at all, and a vendor has no single place to see how they are actually doing on the platform. A read-only aggregate solves both without undoing the isolation fix.

```
vendor_platform_summary (read-only view, not a second authoritative table)
  vendor_id, total_platform_verified_events (summed across companies),
  aggregate_on_time_pct (weighted average), num_companies_worked_with, last_updated
```

This aggregate excludes self-reported data entirely.

It is built only from platform_verified_event_count and the real events behind it — never from a company's halved legacy-import baseline. If a self-reported figure from one company leaked into a shared platform view, the exact cross-company contamination problem the company_id fix just solved would reappear one level up. Restricting the aggregate to platform-verified evidence is precisely why platform_verified_event_count (Section 5.7.2) is useful beyond its original purpose.

Who uses it, and how:

- The vendor sees this on their own dashboard (Part 8.1) — one honest, platform-wide reliability figure, rather than being shown several fragmented private company scores that would not mean anything to them individually.
- A new company with no history with a vendor sees it as informational context on the Quotation Comparison page — e.g. “This vendor has completed 40 verified transactions across 6 other companies, 94% on-time” — displayed to the human, but never fed into that company's own Context Gate calculation. Their gate still runs purely on their own private relationship data (data_volume = 0 for a truly new relationship), keeping the isolation principle intact; the aggregate informs the person, not the algorithm.

- Platform Admin can use it for oversight — spotting a vendor whose reliability is trending badly across the whole platform — separate from Neo4j's fraud-pattern detection, which already operates cross-company by nature (Part 2.6).

This also sets up Phase 3 of the original roadmap cleanly: “vendor reputation travels across companies, with consent” becomes a deliberate later feature — optionally letting a company's own Context Gate factor in this already-existing aggregate, with the vendor's consent — rather than something that needs to be retrofitted after the fact.

To state this without any ambiguity, since it is easy to assume otherwise:

When a brand-new company ranks a quotation from a vendor it has never worked with, the Context Gate (Part 3.3.1) reads only that company's own `vendor_risk_scores` row — which genuinely has `data_volume = 0`. It does not read `vendor_platform_summary`, and it does not borrow confidence from any other company's history with that vendor, no matter how many verified transactions that vendor has completed elsewhere on the platform. The aggregate is visible on the Quotation Comparison page purely as information a human can weigh; the automated ranking itself stays scoped to first-party, verified evidence only, exactly as Section 5.7.1's `company_id` isolation was built to guarantee.

Part 6 — Agent Inventory: What We're Actually Building

6.1 The original 9 agents (from the initial proposal)

The first Veridion proposal document (Section 19) listed nine specialised agents:

- 1. OCR — reads uploaded documents and extracts raw data
- 2. Data Structuring — converts extracted data into a consistent, structured format
- 3. Semantic Matching — compares invoice, PO, and GRN line items for consistency
- 4. Compliance — verifies GST and tax correctness
- 5. Fraud Detection — flags suspicious financial patterns
- 6. Vendor Risk — scores vendor reliability using delivery and dispute history
- 7. Legal Compliance — checks contract terms, regulatory obligations, audit readiness
- 8. Vendor Communication — drafts dispute messages when a mismatch is found
- 9. Decision & Approval Engine — aggregates all agent outputs into the final call

6.2 The 8 agents actually being built, mapped to the original 9

For a 3-person, 6-month build, two of the original nine were deliberately merged rather than kept as fully separate services — a conscious scoping decision, not a scope cut.

Build agent	Maps to original #	What changed
Agent 1 — OCR & Structuring	#1 + #2	Merged — extraction and structuring happen as one LLM call in practice
Agent 2 — Semantic Matching	#3	Unchanged
Agent 3 — Compliance	#4 (+ #7 folded in)	Legal Compliance checks folded in, since #7 was thin on its own
Agent 4 — Fraud Detection	#5	Unchanged
Agent 5 — Vendor Risk Scoring	#6	Unchanged — see Part 5 for its exact update logic
Agent 6 — Quotation Ranking	(new)	Made explicit — was described as a feature in the original proposal but not listed as one of the 9
Agent 7 — Vendor Communication	#8	Unchanged
Agent 8 — Decision & Approval Engine	#9	Unchanged — this is where the Context Gate lives

If the report needs to say “nine agents” to match the original pitch, Legal Compliance can be kept as its own thin agent instead of folding it into Compliance — note this as a deliberate choice either way, not an oversight.

6.3 Where the Context Gate runs — suppliers vs. consumers

The gate itself (the confidence-weighted fusion math) runs in exactly two agents. Every other agent is a supplier — it produces a verdict, a confidence score, and a data_volume, with no awareness of any gate or of any other agent.

Role	Agents	What they do
Suppliers	Agent 2 (Matching), Agent 3 (Compliance), Agent 4 (Fraud), Agent 5 (Vendor Risk)	Each independently produces verdict + confidence + data_volume
Gate consumers	Agent 6 (Quotation Ranking), Agent 8 (Decision Engine)	Each reads multiple suppliers' outputs and runs the weighting formula (Part 3, Section 3.2) to reach one final call

Agent 5 does not use the Context Gate. It only supplies the two numbers — score and data_volume — that Agent 6 and Agent 8 each need in order to run the gate themselves.

6.4 Agent 5 vs Agent 8 vs Agent 6 — the exact relationship

Agent 5 (Vendor Risk)	---> produces: score, data_volume (no gate logic here)
Agent 8 (Decision Engine) reads:	Agent 2's output + Agent 3's output + Agent 4's output + Agent 5's output ---> runs the gate formula once ---> outputs: final_decision
Agent 6 (Quotation Ranking) reads:	Price + Delivery Timeline + Agent 5's output ---> runs the gate formula again, separately, with different inputs ---> outputs: ranked quotation list

The gate formula is written once as a shared function, but called from two different places with two different sets of supplier inputs — this keeps the logic centralised while letting it serve two different decisions.

6.5 Why the 9 agents are still needed even though the PO is system-generated

A natural question: if Veridion generates the Purchase Order itself, correctly, why would any document ever mismatch? The answer is that the PO is only one of five documents in the lifecycle — the two documents that actually get reconciled against it come from outside the system entirely:

Document	Who creates it	Under the system's control?
Requirement	Company, via the platform	Yes
Quotation	Vendor, via the platform (numbers are theirs)	Partially
Purchase Order	Generated by Veridion itself	Yes — always correct
Goods Receipt Note	Warehouse staff, physically counting delivered goods	No — human, real-world
Invoice	The vendor, typically from their own accounting software	No — external, human-entered

A correctly generated PO does not stop a warehouse worker from mis-counting a delivery, and it does not stop a vendor from submitting an invoice with a typo, a billing error, or a deliberate overcharge — both of those documents are created outside Veridion's control. Matching and Compliance exist to catch exactly that gap.

Two agents go further and remain necessary even in a hypothetical world where every invoice matched its PO and GRN perfectly:

- Fraud Detection looks for behavioural patterns across vendors and time — a shell vendor sharing a bank account with another “different” vendor, or one large invoice deliberately split into several smaller ones to dodge an approval threshold. Both can occur with numerically clean, correctly-matching invoices, so no amount of correct PO generation prevents them.
- Vendor Risk Scoring is not a correctness check at all — it is a predictive signal about future reliability, used in ranking a vendor's next quotation. It has nothing to do with whether today's invoice numbers add up.

In one line: Veridion controls the Purchase Order, but not the two documents that actually get reconciled against it — the Goods Receipt Note (a human's physical count) and the Invoice (the vendor's own submission) — so verification remains necessary regardless of how correctly the PO itself is generated.

Part 7 — Handling Legitimate Real-World Variance (Without Weakening Fraud Detection)

Comparing every invoice directly against the original Purchase Order treats three completely normal, honest business situations as if they were mismatches. This section fixes that by changing what gets compared against what — without loosening fraud detection at all.

7.1 Partial delivery — match against the GRN, not the PO

The problem: if a vendor ships less than the full PO quantity in one batch (e.g. 480 of 500 bags, with the remaining 20 to follow), comparing that invoice directly against the PO's 500 looks like a mismatch, even though the vendor billed correctly for exactly what they shipped.

The fix: Semantic Matching (Agent 2) compares an invoice against its own linked Goods Receipt Note, not against the full PO quantity. If the invoice quantity equals the GRN quantity, that is a full, correct match — regardless of how it compares to the PO total.

Whether the PO itself is fully satisfied yet becomes a separate, non-blocking status tracked on the purchase order: pending → partially_fulfilled → fulfilled. This is shown on the PO detail page as information, never treated as a discrepancy or sent to Finance for review.

7.1.1 Optional: capturing an expected date for the remaining quantity

When a shortfall is detected (`received_quantity < agreed_quantity` on that PO), the Warehouse role's GRN entry form conditionally reveals two additional fields — not shown at all until a shortfall is actually detected, so the form stays unchanged for every normal, complete delivery:

- `expected_next_delivery_date` — an optional date picker (“Expected date for remaining quantity”)
- `next_delivery_notes` — an optional short text field, for anything the vendor or driver mentioned at the point of receiving (e.g. “remaining 20 bags, per vendor”)

Both fields are strictly optional and never block submission. If the warehouse staff has no information — the vendor didn't mention anything — they leave both blank and submit exactly as before. If the vendor did volunteer a date, it gets captured for free at the one moment someone is already looking at the shortfall, rather than requiring Procurement to separately chase the vendor for the same information later.

Schema addition to `goods_receipt_notes` (both nullable, both non-blocking):

```
goods_receipt_notes
+ expected_next_delivery_date (nullable date)
+ next_delivery_notes         (nullable text)
```

This connects directly to the existing partial-delivery handling above: the PO detail page can now show not just “partially_fulfilled” but a concrete, human-provided expectation — for example, “480 of 500 received — remaining 20 expected 12 Aug, per warehouse notes.” Procurement sees this without having to independently follow up with the vendor. No other agent or table needs to change: Matching still compares each invoice against its own GRN exactly as before, and these two fields are purely informational, never used as an input to any AI agent or the Context Gate.

7.2 Tax recalculated at invoice time — validate against current rates

The problem: a PO generated weeks before the invoice carries a tax estimate from that earlier date. GST rates, applicable cess, or rounding conventions can legitimately change by the time the invoice is raised — this is routine, not an error by either party.

The fix: Compliance (Agent 3) stops comparing the invoice's tax figure against the PO's stored estimate. Instead it independently recalculates what the tax should be, using the applicable GST rate for that item category as of the invoice date, and checks the invoice against that fresh calculation. The PO's original tax figure is kept only as a reference estimate, never as the basis for a compliance check.

7.3 Multiple partial invoices — cumulative tracking on the PO

The problem: a vendor invoicing in batches (300 now, 200 later) makes each individual invoice look incomplete against the full PO quantity, even though the batches correctly add up over time.

The fix: each individual invoice is still checked against its own linked GRN (per Section 7.1) — never against the full PO. Separately, the PO accumulates a running total (cumulative_invoiced_quantity, cumulative_invoiced_amount) updated with every new invoice against it. Only that cumulative total is ever compared to the PO's agreed total, and only to catch genuine overbilling — not to flag any individual partial invoice as incomplete.

7.4 Why fraud detection stays just as strict

These three fixes only change how legitimate variance is read by the Matching and Compliance agents — they do not touch Fraud Detection (Agent 4), which looks for a different signature entirely:

- A legitimate partial invoice always has a real GRN behind it — a physical delivery, recorded with a timestamp and warehouse sign-off, before the invoice was ever submitted.
- Fraudulent split-billing looks different: invoices submitted without a proportional GRN backing them, or several invoices raised against a single physical delivery event, timed close together, each kept just under the auto-approval threshold.

Fraud Detection continues to run its own independent check — using GRN records as ground truth — regardless of how Matching or Compliance read any individual invoice. Making Matching smarter about legitimate variance does not make the platform more permissive toward fraud; it makes the difference between the two sharper, because genuine partial deliveries stop generating noise that could otherwise bury a real fraud signal.

7.5 Updated schema changes required

These fixes require a small set of additions to the schema defined in Part 2:

Table	New/changed columns
purchase_orders	+ fulfillment_status ('pending'/'partially_fulfilled'/'fulfilled'), + cumulative_invoiced_quantity, + cumulative_invoiced_amount
goods_receipt_notes	already supports multiple GRN rows per po_id, which is what partial deliveries require; + expected_next_delivery_date (nullable date), + next_delivery_notes (nullable text) — both optional, captured per Section 7.1.1
invoices	unchanged — already links to a specific grn_id, which Section 7.1's fix depends on
compliance_checks	+ gst_rate_used, + gst_rate_reference_date (the rate the recalculation was checked against, for audit purposes)
matching_results	+ issue_type ('informational' / 'mismatch' / 'fraud_suspect') — lets the UI distinguish a partial-delivery note from a genuine discrepancy

With issue_type added, the Invoice Review page (page 21) can now show partial deliveries and tax recalculations as quiet, informational notes — while still surfacing a true mismatch or fraud suspicion exactly as before.

Part 8 — System Interfaces: Roles, Purpose & Capabilities

This section supersedes an earlier draft of this part. The earlier version treated Procurement and Finance as two separate top-level interfaces, alongside a separate Warehouse interface. That was structurally wrong: Procurement, Finance, and Warehouse staff all belong to the same organisation and should share one account boundary, not be modelled as unrelated interfaces. The corrected structure below reflects that.

Veridion has three top-level interfaces, still built as one shared React application with role-based routing and RBAC — not separate codebases. Company Profile is a single interface shared by four internal roles.

Interface	Who's inside it	Account boundary
Vendor	External vendor staff	One vendor account per vendor company
Company Profile	Company Admin, Procurement, Finance, Warehouse — all under one company_id	One company account, four internal roles
Platform Admin	Veridion's own platform administrators	Company-agnostic, oversees the whole platform

8.1 Vendor Interface

Who uses it: External vendors — the businesses supplying goods or services (e.g. SteelCorp).

Purpose: everything a vendor needs to interact with a company, from first registering to getting paid.

- Registers the vendor (GST/PAN/business proof upload) and shows live verification status
- Shows open requirements the vendor can quote on, browsable by category
- Lets the vendor submit a quotation (price + delivery timeline) and track its status
- Shows accepted Purchase Orders
- Lets the vendor upload an invoice against a PO/GRN
- Shows live invoice status: submitted → verified → flagged → approved → paid
- Shows any dispute flagged against them, with the AI-drafted explanation, so they can respond
- Shows the vendor's own trust/performance score, aggregated across every company they've worked with on the platform (vendor_platform_summary, Part 5.8) — kept transparent deliberately, so it reads as accountability rather than an opaque judgment, and as one coherent figure rather than several fragmented private company scores
- Tracks payment status and due dates

Why it's separate: the vendor is an outside party and must never see other vendors' quotes, another company's internal data, or anything on the Company Profile or Platform Admin side.

8.2 Company Profile Interface

One interface, one account boundary (company_id), shared by four roles. A company registers once; everyone else on that company's team is added internally afterward, not through a public form (see Section 8.4).

8.2.1 Company Admin

Who: the person who registered the company, or anyone they've promoted to this role.

- Adds and removes team members, assigning each one a role: Procurement, Finance, or Warehouse
- Has visibility across everything Procurement and Finance can see (a company-side oversight role)
- Manages the company's own profile details (name, GSTIN, address, industry type)

Note: this is distinct from Platform Admin (Section 8.3). Company Admin governs one company's own team; Platform Admin governs the whole platform, including vendor trust and Context Gate tuning.

8.2.2 Procurement role

Purpose: managing the requirement-to-purchase-order half of the lifecycle.

- Posts new requirements (e.g. “need 500 bags of cement by the 15th”)
- Views the AI-ranked comparison of all vendor quotations, with plain-English reasoning per vendor
- Accepts a quotation — can override the AI's top pick — which triggers Purchase Order generation
- Views and downloads the generated PO
- Browses the vendor directory, searching by category and reviewing trust scores and history
- Cannot: approve payments, view the Vendor Performance financial dashboard, or resolve disputes

8.2.3 Finance role

Purpose: everything from invoice verification through to payment.

- Sees all invoices pending review, and all flagged invoices
- Reviews an invoice with the AI's full breakdown — the document itself, mismatched fields highlighted using OCR bounding boxes, and the Context Gate's plain-English reasoning
- Reviews and edits the AI-drafted dispute message before sending it to the vendor
- Approves payments
- Views the Vendor Performance dashboard — dispute rate, mismatch type breakdown, resolution time, risk trend
- Cannot: post requirements, select vendors, or generate a PO

8.2.4 Warehouse role — detailed

Purpose: one job — recording what actually arrived. This is deliberately the narrowest role on the platform, and the single source of ground truth the entire AI verification pipeline depends on, including the legitimate-variance handling in Part 7.

Can do:

- View Purchase Orders relevant to receiving — item description, quantity, expected delivery date only
- Create a Goods Receipt Note against a PO: received quantity, received date, warehouse notes
- Flag a discrepancy at the point of receiving if the delivered quantity doesn't match the PO
- View their own GRN submission history

Cannot do:

- Cannot view price, payment terms, or any financial figure on a PO — pricing is not warehouse-floor information
- Cannot view quotations, vendor rankings, or vendor selection
- Cannot view invoices, payments, disputes, or the Vendor Performance dashboard
- Cannot view fraud flags or any Admin-level data
- Cannot edit a PO or a GRN once submitted — corrections go through a Company Admin

How the price restriction is enforced:

Rather than a separate database table, this is enforced at the API layer: the endpoint that serves PO details to the Warehouse role strips the price/amount fields from the response before sending it, using the same

purchase_orders row every other role reads. This keeps one schema and one source of truth, with field-level visibility handled in application code, not duplicated data.

8.3 Platform Admin Interface

Who uses it: Veridion's own platform administrators — not part of any single company.

Purpose: governing the platform itself, across all companies and vendors.

- Reviews the vendor verification queue — sees automated GST/PAN/document checks, gives final human sign-off before a vendor goes live
- Reviews AI-detected fraud flags (shell company matches, split-billing patterns) and marks them resolved or escalated
- Reviews the monthly agent_overrides log and adjusts Context Gate weighting constants if a pattern emerges (Part 3.4)
- Manages platform-wide settings — approval thresholds, notification rules
- Can view (not edit) any company's account for support/audit purposes

Why it's separate from Company Admin: Company Admin manages one company's own team and data; Platform Admin has cross-company authority over vendor trust and the AI system itself — a materially different, higher-trust role that should never be assignable by a company.

8.4 Registration & Login — exactly what's needed, no more

Two public registration flows exist, plus one internal (non-public) flow, plus one shared login used by everyone regardless of role.

Flow	Public?	What it does
Vendor Registration	Yes — anyone can start this	Collects GST/PAN/business proof, creates a vendors row with verification_status = 'pending', triggers Part 6.5's verification checks
Company Registration	Yes — anyone can start this	Creates the companies row and the first user, automatically assigned the company_admin role
Add Team Member	No — internal only, inside Company Profile	Company Admin enters a teammate's email and assigns a role (procurement/finance/warehouse); an invitation is created, the teammate sets a password on accepting it
Login	Shared by everyone	One login page for all roles; on success, the JWT's role claim determines which interface renders

There is deliberately no public registration form for Procurement, Finance, or Warehouse — allowing that would let anyone self-assign access to an existing company's financial or operational data, which is a real security hole, not just an inconvenience.

8.5 Required table changes for this restructuring

This restructuring is a routing and permissions change more than a schema change — the users table already had role and company_id columns — but a few additions are required to make it correct and complete.

Table	Change
users	role column values updated to: 'vendor', 'company_admin', 'procurement', 'finance', 'warehouse', 'platform_admin' (previously just 'vendor'/'procurement'/'finance'/'admin', which conflated Company Admin and Platform Admin into one ambiguous 'admin' value)
companies	+ created_by (FK to users) — records which user originally registered the company and became its first company_admin
invitations (new table)	id, company_id (FK), invited_email, invited_role

	('procurement'/'finance'/'warehouse'), invited_by (FK to users), status ('pending'/'accepted'/'expired'), token, created_at, accepted_at — backs the Add Team Member flow
roles_permissions	re-populated with the corrected role set above; each permission_key (e.g. can_approve_payment, can_select_vendor, can_view_po_price) is mapped only to the roles that should hold it — this is what the API layer checks before returning price fields to the Warehouse role
purchase_orders	no structural change — the price/amount fields already exist; the Warehouse restriction is enforced by the API response layer (Section 8.2.4), not by splitting the table

No changes are required to the procurement lifecycle tables from Part 2.2, the AI/Context Gate tables from Part 2.3, or the Part 7 variance-handling additions — this restructuring only touches identity, access, and how existing data is filtered per role.

Part 9 — Automated Vendor Notification via Voice Call

Today, once Procurement accepts a quotation, the vendor only finds out by logging into the platform and checking. This section adds an automated notification the moment a quotation is accepted, closing that gap.

9.1 Trigger point

The same event that generates the Purchase Order — Part 4, Step 7 (“Vendor selected”) — also fires an outbound notification to the vendor. It is a side-effect of that action, not a separate workflow: the human decides which vendor to accept, and the system both generates the PO and informs the vendor automatically.

9.2 Provider choice — OmniDimension

OmniDimension, a no-code voice AI agent platform, is used for this feature rather than assembling telephony, speech-to-text, and text-to-speech separately (the approach a lower-level provider such as Twilio would require). Reasons this fits Veridion specifically:

- The call's purpose is described in plain language when configuring the agent (“inform the vendor their quotation was accepted, direct them to their dashboard”), rather than hand-written call scripts
- It connects to any custom API via webhook — the backend fires a webhook the instant a quotation is accepted, and OmniDimension places the call
- Multilingual support, including Hindi and Tamil, fits Veridion's positioning as GST-native for Indian mid-size companies
- It replaces a lower-level telephony provider entirely for this feature — one account and one set of credentials instead of separately wiring together call routing, speech-to-text, and text-to-speech

Cost note for the report: OmniDimension is free to start with no card required, but runs on a credit/minute-based system beyond the initial free credits, with paid plans afterward. For a student project's development and demo scale (a small number of test vendors, a handful of calls during building and the presentation itself), this comfortably fits inside the free tier. The accurate framing is “low-cost, generous free tier for development and demo scale,” not “completely free at any volume.”

9.3 Required configuration (API keys)

OMNIDIMENSION_API_KEY	-- account API key
OMNIDIMENSION_AGENT_ID	-- the configured voice agent's ID
OMNIDIMENSION_OUTBOUND_NUMBER_ID	-- the calling number assigned to the account

These are stored as environment variables, never committed to the repository, matching the existing convention for the platform's other credentials (Groq API key, database connection strings).

9.4 Message content

The call stays short and purely informational — it never requires a response over the call itself:

“This is an automated message from Veridion on behalf of [Company]. Your quotation for [item] has been accepted. A purchase order has been generated and is available on your Veridion vendor dashboard. Thank you.”

9.5 Non-blocking design rule

This notification must never delay or block Purchase Order generation itself. If OmniDimension's API is unreachable, or the vendor's stored phone number is invalid, PO generation still completes normally — the call is a courtesy layer on top of an already-completed action, not a dependency of it. The outbound call is fired

asynchronously after the PO row is committed, with its own independent success/failure tracked in `outbound_notifications` (Section 9.6), never inside the same transaction as PO creation.

9.6 Schema additions

Two changes support this feature, both already reflected in Part 2:

- vendors gains `phone_number` and `preferred_notification_channel` ('voice_call'/'sms'/'whatsapp'/'app_only', default 'sms') — collected at vendor registration, so a vendor can opt for a text instead of a call if they prefer
- A new `outbound_notifications` table logs every attempt — `vendor_id`, `reference_type`, `reference_id`, `channel`, `status`, `provider_message_id`, `sent_at` — giving an audit trail (useful if a vendor claims they were never informed) and a ready extension point for other automated notifications later (payment reminders, dispute alerts) without inventing a new table each time

Summary

Nineteen-plus relational tables with JSONB columns for flexible AI outputs, files kept on disk with paths stored in PostgreSQL, three deliberate database transactions at the points where atomicity actually matters (payment approval, quotation acceptance, vendor verification), RabbitMQ for parallel agent execution, Neo4j for graph-based fraud detection, a Context Gate deliberately implemented as a transparent, auditable formula rather than a trained model given the platform's lack of historical training data and its need for plain-English explainability (Part 3.5), a worked example showing how a new vendor's price advantage is weighed fairly against an established vendor's track record (Part 3.3.1), a stated principle that the system's confidence is always shown beside its decision and the company always makes the final vendor/approval choice (Part 3.6), an event-triggered running-average update engine for vendor risk metrics with an explicit no-prefilling, cold-start-by-design principle and an optional discounted legacy-import path for pre-platform vendor history (Part 5.7), vendor_risk_scores scoped per company from the outset so one company's experience with a vendor never silently affects another's, alongside a read-only, verified-only platform-wide aggregate for the vendor's own view and for new companies evaluating an unfamiliar vendor (Part 5.8), eight agents mapped cleanly against the original nine with the Context Gate running in exactly two of them (Part 6), a clear account of why verification remains necessary even though the Purchase Order itself is generated correctly by the system (Part 6.5), explicit handling for legitimate real-world variance that keeps honest business situations from being treated as mismatches without loosening fraud detection (Part 7), three top-level interfaces — Vendor, Company Profile (housing Company Admin, Procurement, Finance, and Warehouse as internal roles under one account boundary), and Platform Admin — with two public registration flows, one internal team-invite flow, and one shared login, each isolating a distinct kind of risk (Part 8), and a non-blocking, OmniDimension-based automated voice notification to the vendor the moment their quotation is accepted (Part 9).